

Integration Blueprint for Office Ticket OCR to SharePoint and On-Premises Export Workflow

Executive Summary

The transition from analog, paper-based scale-house operations to a digitized, automated back-office ecosystem requires a highly resilient, event-driven architectural framework. This comprehensive technical report details the exact integration blueprint for implementing a Phase 1 office-side prototype. The target workflow encompasses the ingestion of ticket images, Optical Character Recognition (OCR) extraction and field normalization, a human-in-the-loop (HITL) review queue for exception handling, asynchronous data writes to Microsoft SharePoint and Excel Online, automated invoice PDF generation, and secure flat-file exports to an on-premises server folder.

This analysis benchmarks TruckIT as the primary workflow standard against comparable heavy building materials platforms, including HaulHub, Command Alkon (Connex), and Tread. Furthermore, the report establishes robust mitigation strategies for complex implementation constraints, specifically addressing Microsoft Graph API concurrency limitations, HTTP 423 file-locking exceptions, HTTP 429 throttling limits, and the operational limitations of the Microsoft On-Premises Data Gateway.

Architecture Options with Tradeoffs

The implementation of the Phase 1 prototype requires selecting a core integration topology to orchestrate the flow of data from the OCR extraction engine to the on-premises enterprise resource planning (ERP) system. The analysis presents three distinct architectural models, evaluating their respective tradeoffs in the context of latency, engineering overhead, and hybrid cloud constraints.

Option A: Serverless PaaS Orchestration

This architecture utilizes Platform-as-a-Service (PaaS) orchestration engines, such as Azure Logic Apps or Microsoft Power Automate, as the central nervous system of the workflow. The OCR service, powered by Azure AI Document Intelligence, extracts data and pushes JSON payloads to the orchestrator. The orchestrator subsequently manages the human review queue, writes normalized records to SharePoint via the Microsoft Graph API, generates invoices using the Word Online (Business) connector, and executes the on-premises file drop via the Microsoft On-Premises Data Gateway.

The primary advantage of this approach is the rapid acceleration of the pilot timeline, leveraging native Microsoft 365 (M365) integrations without the need to provision dedicated compute clusters. However, the tradeoffs are significant regarding licensing and operational limits. The architecture incurs premium licensing costs, specifically for the Microsoft Word

template population action, which requires a Power Automate Premium license¹ (URL: https://www.lindsaytshelton.com/blog/20250112populating_a_word_template_for_an_easy_win/, Confidence: High). Furthermore, the On-Premises Data Gateway enforces strict file size limitations for the File System connector, capping file creation operations at 20 megabytes and general operations at 30 megabytes³ (URL: <https://learn.microsoft.com/en-us/connectors/filesystem/>, Confidence: High).

Option B: Custom Event-Driven Microservice

The second architectural option relies on a custom middleware service, typically built in C#/ .NET or Node.js, deployed on Azure Kubernetes Service (AKS) or Azure App Service. This custom middleware acts as a webhook receiver, consuming events from the OCR engine or the ticketing vendor's API. It validates the JSON schema, executes highly parallelized Microsoft Graph API calls using the native SDK, and pushes flat files to the on-premises network via a Site-to-Site VPN, Azure ExpressRoute, or a secure SFTP drop.

This approach affords absolute programmatic control over concurrency limits, Entity Tag (ETag) management, and exponential backoff strategies, effectively circumventing the rigid file size limitations of Power Automate. The microservice architecture allows for sophisticated in-memory transformations and localized caching. The tradeoff involves a substantially higher engineering overhead, extending the critical path of the 4-6 week pilot. Additionally, it introduces the operational burden of maintaining, patching, and securing the hosting infrastructure, as well as managing the network routing required for the on-premises export phase.

Option C: Vendor-Native Agent Integration

The third architecture minimizes custom cloud middleware by relying entirely on the ticketing vendor's proprietary synchronization agents. For example, systems like HaulHub utilize a FedRAMP and PCI-certified lightweight agent known as the HaulHub Connector (HHC) Atom⁵ (URL: <https://knowledge.haulhub.com/hubfs/connect/Data%20Integration%20Process.pdf>, Confidence: High). This read-only Windows service runs on the on-premises network, continuously polling the vendor API and pulling data directly into the local database over a secure outbound SSL connection on TCP port 443. SharePoint and Excel updates are subsequently handled by an on-premises scheduled task that pushes data back to the M365 cloud environment.

While this option eliminates the need for complex cloud-to-ground firewall configurations and leverages highly secure, purpose-built agents, it introduces severe architectural constraints. It creates substantial vendor lock-in and severely limits the ability to inject custom human-in-the-loop (HITL) review steps into the cloud workflow prior to database ingestion. Furthermore, routing cloud-native M365 integrations through an on-premises bottleneck contradicts modern cloud-first design principles, making this the least optimal choice for a flexible prototype.

Architecture Comparison Matrix

Feature	Option A: Serverless PaaS	Option B: Custom Microservice	Option C: Vendor-Native Agent
Primary Technology	Power Automate / Logic Apps	C# .NET / Node.js on Azure	Vendor specific (e.g., HHC Atom)
Engineering Effort	Low (Low-Code/No-Code)	High (Full SDLC)	Low (Vendor Managed)
On-Premises Routing	MS On-Premises Data Gateway	Site-to-Site VPN / SFTP	Proprietary Outbound Agent
File Size Constraints	20 MB Creation / 30 MB General	Bounded only by RAM/Network	Dependent on Vendor Specs
M365 Integration	Native Connectors (Premium)	Microsoft Graph API SDK	Custom On-Prem Scripting
Recommendation	Primary Choice for Pilot	Optimal for Phase 2 Scale	Not Recommended for Prototype

Integration Surface Map per Platform

An exhaustive evaluation of the integration surfaces for the primary benchmark target, TruckIT, alongside comparable systems—HaulHub, Command Alkon, and Tread—reveals varying degrees of API maturity, authentication standards, and operational constraints.

TruckIT (Workflow Benchmark)

TruckIT operates as the primary benchmark for heavy hauling logistics, offering automated e-ticketing ("AirTicket") and OCR ticket digitization ("Automatic-PIC" or ATP), which utilizes computer vision and machine learning to extract data from handwritten or printed tickets⁶ (URL: <https://www.worktruckonline.com/news/blue-max-trucking-moves-to-digital-ticketing>, Confidence: High). The platform exposes a RESTful API designed to facilitate quoting, booking, and shipment tracking⁹ (URL: <https://developers.truckit.net/>, Confidence: High).

The API requires secure authentication via OAuth2 protocols, mandating specific HTTP headers for machine-to-machine interactions. Integration requests must include Authorization, x-api-key, x-customer-id, and x-customer-secret headers¹⁰ (URL: <https://developers.truckit.net/api/endpoints/booking>, Confidence: High). The data contracts

require JSON payloads containing UUID references, contact structures, and nested arrays for line items. TruckIT also supports an event-driven architecture via webhooks, allowing developers to programmatically register, update, and delete webhook endpoints via POST, PATCH, and DELETE requests to receive real-time notifications for status changes¹⁰ (URL: <https://developers.truckit.net/api/endpoints/booking>, Confidence: High).

HaulHub

HaulHub provides the EDOT Portal and enforces a highly structured, rigid public ticket ingest mechanism. The platform exposes a REST API that mandates JSON payloads submitted via POST requests to the `/integrations/api/v1/tickets` endpoint¹¹ (URL: <https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>, Confidence: High). Authentication relies on a supplier-specific access token passed via the `x-hh-api-token` header, alongside a standard `Content-Type: application/json` declaration¹¹ (URL: <https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>, Confidence: High).

HaulHub's integration surface is notable for its strict payload validation rules. Aggregate and asphalt tickets are capped at 100 ticket objects per request, while ready-mix concrete tickets are strictly capped at 25 per request; exceeding these limits triggers a hard HTTP 400 Bad Request error¹¹ (URL:

<https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>, Confidence: High). Furthermore, HaulHub supports outgoing webhooks managed through a GraphQL server interface, allowing event subscriptions for actions such as `ticket.created`, `ticket.updated`, and `ticket.voided`¹² (URL: <https://help.tacinsight.com/fast-weight-knowledge-base/references/integrations/haulhub>, Confidence: High).

Command Alkon (Connex)

Command Alkon's Connex platform operates within the Command Cloud, an Amazon Web Services (AWS) hosted infrastructure boasting SOC 1/2 and NIST certifications¹³ (URL: <https://commandalkon.com/command-cloud/>, Confidence: High). The integration surface exposes a library of vetted APIs categorized by domain, including the Order & Ticket Management API, Mix Design Management API, Tracking & Telematics API, and Billing & Invoicing API¹⁴ (URL: <https://commandalkon.com/products/apis/>, Confidence: High).

Access to the Command Cloud API relies on their Connected Partner Program and a dedicated Developer Portal, which enforces cyber and cloud security controls, including multi-factor authentication (MFA) and single sign-on (SSO) frameworks¹³ (URL:

<https://commandalkon.com/command-cloud/>, Confidence: High). The platform acts as a centralized data lake, supporting the pseudonymization of data subjects to comply with global privacy regulations¹⁶ (URL: <https://apac.cadev.deployedweb.com/product-privacy/>, Confidence: High). The underlying architecture heavily utilizes serverless components, such as AWS DynamoDB for transactional storage and Rockset for real-time analytics indexing¹⁷ (URL: <https://medium.com/rocksetcloud/bringing-real-time-analytics-to-construction-logistics-at-co>

mmand-alkon-e85bdb477442, Confidence: Medium).

Tread

Tread provides an AI-native Software-as-a-Service (SaaS) platform built on a Ruby on Rails backend, heavily utilizing JavaScript/React and GraphQL schemas for its front-end and API interactions¹⁸ (URL: <https://www.tealhq.com/job/ai-native-forward-deployed-engineer>, Confidence: High). The platform exposes open APIs supporting both REST and GraphQL architectures, enabling deep technical integrations with telematics systems, Electronic Logging Devices (ELDs), and accounting software such as QuickBooks and Sage Intacct¹⁹ (URL: <https://tentrucks.com/blog/best-trucking-software-2026-tech-stack>, Confidence: High). Tread's data model heavily incorporates geofencing, route playback, and time-tracking data, digitizing the traditional paper ticketing process to eliminate manual errors and accelerate invoice processing²⁰ (URL: <https://softwarefinder.com/fleet-management-software/tread>, Confidence: High). The technical ecosystem is designed for rapid iteration, allowing developers to read GraphQL schemas, debug data pipelines, and execute complex database migrations to support custom workflows¹⁸ (URL: <https://www.tealhq.com/job/ai-native-forward-deployed-engineer>, Confidence: High).

Platform Integration Matrix

Platform	API Type	Auth Method	Rate Limits / Caps	Webhook Support	Primary Use Case
TruckIT	REST	OAuth2, custom headers	Unknown (Requires Demo)	Yes (POST, PATCH, DELETE)	Automated dispatch, OCR digitization
HaulHub	REST	API Key (x-hh-api-token)	100 Agg / 25 Ready Mix per POST	Yes (GraphQL configured)	DOT compliance, e-Ticketing ingest
Command Alkon	REST / Open API	API Portal, SSO, MFA	Real-time (DynamoDB backed)	Developer Portal Restricted	Enterprise ERP, broad material types
Tread	REST / GraphQL	Unknown (Requires Demo)	Unknown (Requires Demo)	Unknown (Requires Demo)	Geofencing , QuickBooks integration

Data Contract Details and Validation Rules

The transition from OCR extraction to structured database records mandates the establishment of rigid data contracts. Based on the stringent industry standards defined by HaulHub and TruckIT, the canonical ticket object must support a highly structured schema with predefined validation rules and error responses.

Ticket Object Fields and Normalization

A normalized OCR schema requires the extraction and transformation of specific canonical fields from the physical ticket image. The foundational structure must differentiate between aggregate/asphalt materials and ready-mix concrete, as the data requirements diverge significantly.

For standard aggregate and asphalt tickets, the required fields include identifiers such as `slip_number` (String or Integer) and `dot_code` (String). Time tracking demands absolute precision, necessitating the `dispatched_at` timestamp formatted in ISO 8601 with an IANA-defined timezone declaration (e.g., 2023-07-10T07:40:00.000-05:00 and America/New_York)¹¹ (URL:

<https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>,

Confidence: High). Material weights must be captured across multiple properties:

`gross_amount`, `tare_amount`, and `net_amount` as floating-point integers, paired with their respective Unit of Measurement (`quantity_uom_code`), which is typically normalized to "TON" or "lbs"¹¹ (URL:

<https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>,

Confidence: High). Categorical data encompassing the `client_id`, `job_number`, `material_number`, and the specific Job Mix Formula (`product_code`) must also be explicitly parsed¹¹ (URL:

<https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>,

Confidence: High).

Ready-mix concrete payloads require deeper nested arrays. The contract must include a `ticket_products` sub-object defining the `product_code`, `product_uom` (e.g., "yd^3" for cubic yards), and the `application_type` set to "primary"¹¹ (URL:

<https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>,

Confidence: High). Furthermore, a `batch_weights` array is required to document theoretical versus actual quantities, moisture percentages, and water-cement ratios

(`target_water_cement_ratio`, `batched_water_cement_ratio`)¹¹ (URL:

<https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>,

Confidence: High).

Duplicate Detection Approach

Processing physical tickets via mobile OCR frequently results in duplicate submissions, as drivers or dispatchers may capture the same document multiple times. To maintain data integrity, the middleware must implement a deterministic duplicate detection approach based on idempotency keys.

The most robust strategy involves generating a cryptographic hash—such as SHA-256—comprising a concatenated string of the `slip_number`, the `plant_id`, and the `dispatched_at` timestamp. This hash serves as the unique identifier within the middleware's operational database. Upon receiving a new payload, the system queries the database for this specific hash; if a collision occurs, the system safely ignores the subsequent payload or updates it based on predefined conflict resolution rules, ensuring idempotent operations across the API surface.

Validation Rules and Error Responses

API endpoints processing the normalized data must enforce strict validation rules. If the JSON syntax is malformed, or if required root-level properties are missing, the system must immediately reject the payload with an HTTP 400 Bad Request error¹¹ (URL: <https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>, Confidence: High). Unauthorized access attempts lacking valid tokens or correct headers must yield an HTTP 401 Unauthorized response¹¹ (URL: <https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>, Confidence: High).

Modern platforms implement a partial success model for batch processing. If a batch of 100 tickets contains one malformed record, the API returns an HTTP 200 OK status, confirming the successful ingestion of 99 records. The response payload includes a rejected array detailing the failed index and the specific validation error reason (e.g., `{"field": "contract_number", "reason": "invalid_value"}`)¹¹ (URL: <https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>, Confidence: High). This mechanism prevents a single data entry error from halting entire logistical pipelines.

Correction and Audit Trail Capability

State Departments of Transportation (DOTs) and enterprise auditing frameworks strictly mandate continuous, immutable audit logs. Any modifications made to a ticket—whether through an automated normalization script or manually via the human-in-the-loop review queue—must be exhaustively tracked.

The data contract must incorporate an `audit_log` nested array. This array records every modification, requiring the `field_name` that was altered, an ISO timezone-aware timestamp, the `from_value` prior to the change, and the `to_value` after the change¹¹ (URL: <https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>, Confidence: High). Additionally, the system must document the exact database host executing the transaction (`log_source_host`) and the identity of the actor (`performed_by_name`), distinguishing between system accounts and specific human reviewers¹¹ (URL: <https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>, Confidence: High).

Spreadsheet Integration Specifics (SharePoint and

Excel Online)

Writing high-velocity, structured ticket data into Microsoft 365 (M365) introduces profound architectural challenges. The integration relies entirely on the Microsoft Graph API, which enforces strict concurrency limits, aggressive throttling algorithms, and complex file-locking mechanisms that must be programmatically managed.

File Locking Behavior and Concurrency Concerns

When writing records to Excel files hosted in SharePoint or OneDrive, the Graph API frequently encounters an HTTP 423 Locked or HTTP 409 Conflict error. This phenomenon occurs when a human user has the target Excel file actively open in the desktop application, initiating a shared or exclusive co-authoring lock that blocks backend API writes²¹ (URL:

<https://stackoverflow.com/questions/61554807/microsoft-update-driveitem-api-throwing-locked-exception-if-the-file-is-open-in>, Confidence: High).

To resolve this bottleneck, developers must inject specific HTTP headers into the API request to bypass the client-side locks. The application must append `Prefer: bypass-shared-lock and/or bypass-checked-out` in the header of the REST call. This header instructs the Graph API to override the co-authoring session and commit the payload directly to the file, circumventing the HTTP 423 exception²¹ (URL:

<https://stackoverflow.com/questions/79749824/unable-to-update-or-delete-sharepoint-library-record-when-file-is-locked>, Confidence: High).

SharePoint ETags and Data Conflicts

When interfacing directly with SharePoint Lists rather than Excel files, optimistic concurrency control is enforced through the use of HTML Entity Tags (ETags)²⁵ (URL:

<https://espc.tech/learning-hub/blog/handling-etag-value-on-sharepoint-rest-api-updates/>, Confidence: High). An ETag represents the specific version of a resource at the exact moment a GET request is executed.

To update a list item via a POST, PATCH, or MERGE request, the application must pass the previously retrieved ETag within the `If-Match` HTTP header²⁵ (URL:

<https://dynamicservicesgroup.com/2025/12/30/sharepoint-rest-apis-with-http-actions-in-power-automate/>, Confidence: High). If the ETag provided by the application does not match the server's current ETag—indicating that another user or process modified the record in the interim—the Graph API rejects the update and returns an HTTP 412 Precondition Failed error²⁸

(URL: <https://learn.microsoft.com/en-us/graph/errors>, Confidence: High). If the architectural design dictates that the automated OCR pipeline should always overwrite existing data regardless of human modifications, the application can forcefully bypass version checking by passing the wildcard string `If-Match: *`²⁷ (URL:

<https://sites.google.com/site/icsharepoint2010/desarrollo/using-the-rest-interface>, Confidence: High).

Retry and Error Handling for Partial Writes

The Microsoft Graph API aggressively throttles calling applications to maintain service stability

across the multi-tenant architecture. When usage limits are exceeded, the API returns an HTTP 429 Too Many Requests or an HTTP 503 Service Unavailable status code²⁸ (URL: <https://learn.microsoft.com/en-us/graph/throttling>, Confidence: High). Crucially, every 429 and 503 response includes a Retry-After header, which specifies the exact number of seconds the client application must wait before submitting another request³¹ (URL: <https://learn.microsoft.com/en-us/graph/throttling>, Confidence: High). The integration middleware must implement an exponential backoff retry strategy that parses and strictly honors this Retry-After integer. Continuous polling, rapid retries, or ignoring the delay header will exacerbate the throttling penalty, causing the requests to accrue against the tenant's usage limits, which can ultimately lead to a temporary block on the offending application³¹ (URL: <https://learn.microsoft.com/en-us/sharepoint/dev/general-development/how-to-avoid-getting-throttled-or-blocked-in-sharepoint-online>, Confidence: High).

Mapping Pattern from OCR Schema to Spreadsheet Columns

The extraction of data via Azure AI Document Intelligence requires a precise mapping mechanism to transform the OCR output into the canonical model, which is then mapped to the respective SharePoint List columns or Excel tables, and finally to the generated Invoice fields. The prebuilt-invoice model returns a structured JSON hierarchy containing nested fields and tables³³ (URL: <https://lobehub.com/skills/haniakrim21-everything-claude-code-azure-ai-document-intelligence-dotnet>, Confidence: High).

Raw OCR Field (Azure AI)	Canonical Object Model	SharePoint / Excel Column	Invoice Field Mapping	Data Type
VendorName	supplier	Supplier_Name	Billed_From_Name	String
InvoiceTotal / NetWeight	net_amount	Net_Tonnage	Qty_Billed	Float
Items[x].Description	line_item_description	Material_Desc	Line_Item_Description	String
InvoiceDate / Time	dispatched_at	Dispatch_Time stamp	Delivery_Date	ISO 8601 Datetime
CustomString1	slip_number	Ticket_Number	Reference_Ticket	String

CustomString2	job_number	Project_Code	Project_Refere nce	String
---------------	------------	--------------	-----------------------	--------

Invoice Generation Specifics

Automating the generation of invoice PDFs directly from the normalized OCR data significantly accelerates the order-to-cash lifecycle, removing administrative bottlenecks and reducing manual billing errors.

Native Invoice Templates vs. Custom PDF Generation

Utilizing Microsoft Power Automate to generate invoices natively offers a streamlined, low-code approach through the "Populate a Microsoft Word template" action. However, technical constraints dictate the implementation limits. The Word Online (Business) connector is explicitly classified as a Premium connector, requiring the service account executing the flow to possess a Power Automate Premium license¹ (URL: <https://learn.microsoft.com/en-us/answers/questions/5827119/power-automate-power-apps-premium-vs-populate-word>, Confidence: High).

The template formatting requires meticulous configuration. The underlying Microsoft Word document must utilize the Developer tab to insert "Plain Text Content Controls" linked to specific metadata fields. The Power Automate connector fundamentally cannot parse or interact with "Rich Text Content Controls," legacy form controls, or nested image content controllers¹ (URL: <https://learn.microsoft.com/en-us/connectors/wordonlinebusiness/>, Confidence: High). Furthermore, accommodating line items from a ticket requires the implementation of a "Repeating Section Content Control" to generate dynamic tables³⁶ (URL: <https://learn.microsoft.com/en-us/connectors/wordonlinebusiness/>, Confidence: High).

For organizations aiming to bypass Premium licensing requirements, a custom generation pipeline provides a viable alternative. This architecture involves generating a raw HTML string containing the invoice data structure within standard Power Automate actions, saving the string as a .html file in OneDrive, and utilizing the standard "Convert file" action to transform the HTML markup into a finalized PDF document³⁷ (URL: https://www.reddit.com/r/PowerApps/comments/1q5gy2y/populating_word_doc_template_without_needing/, Confidence: High). Alternatively, data can be passed to a SharePoint Document Library where columns are linked directly into Word document fields, creating a PDF output without Premium connectors³⁸ (URL:

<https://espc.tech/learning-hub/blog/fill-word-templates-without-premium-connectors-in-power-automate-guide-%F0%9F%A4%96/>, Confidence: Medium).

Numbering Rules, Tax Handling, and Versioning

The billing middleware must assume absolute governance over invoice numbering rules to guarantee sequential integrity, which is a mandatory requirement for ERP synchronization and financial auditing. Tax calculations must be strictly decoupled from the initial OCR extraction

phase, as raw ticket data rarely contains accurate jurisdictional tax information. Instead, the architecture should rely on a centralized, cached tax lookup table linked to the `client_id` or `job_number` to cross-reference and inject accurate tax logic during the generation phase. When an invoice requires regeneration due to an audit correction or a change in payload data, the system must never overwrite the original document. Overwriting destroys the financial audit trail. The logic must append a versioning suffix to the file name (e.g., INV-1024-v2) and update the corresponding SharePoint metadata to reflect the revised status, preserving all historical context.

On-Premises Export Options and Hybrid Constraints

Synchronizing cloud-processed ticket data with legacy on-premises infrastructure is the final, critical segment of the integration blueprint. This requires establishing a secure, reliable, outbound-only tunnel to bypass corporate firewalls without compromising network integrity.

Secure Transfer Patterns and the Data Gateway

The optimal secure transfer pattern for Microsoft-centric architectures is the Microsoft On-Premises Data Gateway. The gateway operates as a lightweight Windows service installed on a server within the corporate network. It establishes a highly secure, persistent, outbound-only WebSocket connection to the Azure Service Bus (Azure Relay) utilizing standard TCP ports 443, 5671, 5672, and 9350 through 9354³⁹ (URL:

<https://ve3.global/blog/the-on-premises-data-gateway--the-most-underestimated-dependency-in-every-power-platform-programme>, Confidence: High). Because the communication flows exclusively outbound, network administrators do not need to open any inbound firewall ports, drastically reducing the external attack surface.

When the Power Automate cloud service initiates a request to write a CSV file to an on-premises folder, the request and encrypted credentials are pushed to the Azure Relay. The gateway polls the relay, pulls the request, decrypts the credentials locally, executes the file write to the local server, and spooling the response back to the cloud⁴¹ (URL:

<https://learn.microsoft.com/en-us/data-integration/gateway/service-gateway-onprem-indepth>, Confidence: High).

Operational Constraints and File Limits

While the On-Premises Data Gateway provides seamless connectivity, it imposes severe operational constraints regarding payload size. The "File System" connector operating through the gateway restricts the maximum file size for a "create file" operation to a hard limit of 20 megabytes, while general operations are capped at 30 megabytes³ (URL:

<https://learn.microsoft.com/en-us/connectors/filesystem/>, Confidence: High). If a daily scheduled export aggregates a large volume of CSV data exceeding this threshold, the middleware must implement pagination or chunking logic to partition the export into smaller, compliant batch files prior to transmission. Furthermore, the gateway enforces a strict connection timeout limit of 30 seconds⁴ (URL:

<https://learn.microsoft.com/en-us/connectors/filesystem/>, Confidence: High).

High Availability and Clustering Architecture

Deploying a single gateway server for production workloads introduces a critical single point of failure into the architecture. To mitigate this risk, infrastructure teams must deploy a high-availability gateway cluster. A cluster consists of multiple gateway instances (up to a maximum of 10 nodes) registered under the same logical cluster name and sharing identical data source definitions and recovery keys⁴⁰ (URL:

<https://learn.microsoft.com/en-us/data-integration/gateway/service-gateway-high-availability-clusters>, Confidence: High).

Cluster traffic routing can be dynamically configured by administrators. In a "Round Robin" configuration, requests are distributed evenly across all active cluster members to maximize throughput and load balance concurrent refresh operations⁴⁰ (URL:

<https://powerbiconsulting.com/blog/power-bi-gateway-architecture-high-availability-2026>, Confidence: High). Alternatively, a "Failover" configuration routes all traffic to the primary node; if the primary node goes offline, the traffic automatically fails over to the secondary node, minimizing the overall resource footprint while maintaining availability⁴⁰ (URL:

<https://powerbiconsulting.com/blog/power-bi-gateway-architecture-high-availability-2026>, Confidence: High). Furthermore, administrators can throttle resource usage by defining CPU and memory limits (`CPUUtilizationPercentageThreshold` and

`MemoryUtilizationPercentageThreshold`) to prevent individual gateway machines from becoming overwhelmed by large data sets⁴² (URL:

<https://learn.microsoft.com/en-us/data-integration/gateway/service-gateway-high-availability-clusters>, Confidence: High).

Implementation Constraints

Deploying the prototype demands rigorous adherence to M365 security paradigms, network prerequisites, and project management timelines to ensure a successful integration.

Required Permissions and Scopes in Microsoft 365

Historically, applications interfacing with SharePoint via the Graph API relied on broad, overarching permission scopes such as `Sites.ReadWrite.All`, granting the application unfettered access to every site collection across the entire M365 tenant⁴³ (URL:

<https://medium.com/@kathanmecii/moving-from-ac-s-password-based-sharepoint-access-to-sites-selected-in-microsoft-graph-bfcdc5f77dd1>, Confidence: High). This violates the principle of least privilege, expanding the blast radius significantly if the service principal is compromised.

The implementation must exclusively utilize the `Sites.Selected`, `Lists.SelectedOperations.Selected`, or `Files.SelectedOperations.Selected` granular permission scopes⁴⁴ (URL: <https://learn.microsoft.com/en-us/graph/permissions-selected-overview>, Confidence: High). When these scopes are assigned in Microsoft Entra ID, the application initially possesses zero access to any data. An M365 administrator must subsequently execute a specific POST request to the <https://graph.microsoft.com/v1.0/sites/{site-id}/permissions> endpoint, explicitly assigning the write role to the application's Client ID for the designated

target site or list⁴⁴ (URL:

<https://medium.com/@kathanmecii/moving-from-ac-password-based-sharepoint-access-to-sites-selected-in-microsoft-graph-bfcdc5f77dd1>, Confidence: High).

Pilot Timeline and Critical Path Dependencies

A typical pilot execution spans a 4-to-6-week timeframe, governed by several critical path dependencies that frequently bottleneck implementation. The primary dependency is the procurement, provisioning, and firewall configuration of the on-premises server intended to host the data gateway³⁹ (URL:

<https://ve3.global/blog/the-on-premises-data-gateway--the-most-underestimated-dependency-in-every-power-platform-programme>, Confidence: High). Engaging the infrastructure and security teams to approve outbound port rules and provision service accounts must occur on day one to prevent deployment delays. Secondly, securing Power Automate Premium licenses and coordinating with M365 global administrators to grant the Sites.Selected Graph API permissions dictates the pace of the cloud workflow deployment.

Sequence Diagram for Recommended Flow

The following textual sequence diagram illustrates the optimal execution path from the physical ticket capture to the final ERP synchronization.

1. **Event Trigger:** A driver or scale-house operator utilizes a mobile application to upload a physical ticket image to Cloud Storage.
2. **OCR Extraction:** An automated webhook triggers Azure AI Document Intelligence, invoking the prebuilt-invoice or a custom extraction model to parse text, bounding regions, and tabular structures.
3. **Validation & Routing:** The integration middleware evaluates the confidence scores returned by the AI model.
 - *If the confidence score is below 0.85:* The payload is routed to a Dead-Letter Queue (DLQ) tied to a Human-in-the-Loop (HITL) manual review interface.
 - *If the confidence score is high:* The payload proceeds automatically to Step 4.
4. **Deduplication Check:** The middleware generates a cryptographic hash (slip_number + dispatched_at) and queries the operational database to guarantee idempotency.
5. **M365 Synchronization:**
 - The middleware executes a POST request to the Microsoft Graph API, committing the normalized data to the targeted SharePoint List.
 - *Exception Handling:* If an HTTP 429 is received, the script delays execution based on the Retry-After header. If an HTTP 423 is received, the request is re-issued with the Prefer: bypass-shared-lock header.
6. **Invoice Generation:** A Power Automate flow, triggered by the creation of the new SharePoint row, populates a designated Microsoft Word Template, converts the output to PDF, and saves the final invoice to a SharePoint Document Library.
7. **On-Premises Export:** A nightly batch job aggregates newly created records into a CSV payload and utilizes the Microsoft On-Premises Data Gateway to write the file directly to the local ERP server, applying chunking logic if the payload exceeds 20 megabytes.

Failure-Mode and Recovery Table

The resilience of the architecture depends on proactive detection and automated recovery algorithms for all anticipated failure states.

Error / Failure Mode	Detection Mechanism	Recovery Action
OCR Hallucination / Low Confidence	Confidence score < 0.85 returned by Azure AI Document Intelligence API.	Route payload to a Dead-Letter Queue (DLQ) attached to a Human-in-the-Loop (HITL) manual review dashboard.
Graph API Throttling	HTTP 429 Too Many Requests or 503 Service Unavailable returned by Microsoft Graph ³¹ .	Parse the integer in the Retry-After HTTP response header. Suspend worker thread execution for the exact specified duration before retrying the payload ³² .
File Lock / Concurrency Conflict	HTTP 423 Locked or 409 Conflict encountered during an Excel or SharePoint update attempt ²¹ .	Re-issue the exact request, injecting the Prefer: bypass-shared-lock HTTP header to override client-side co-authoring locks ²³ .
ETag Versioning Mismatch	HTTP 412 Precondition Failed during a SharePoint List PATCH operation ²⁸ .	Re-fetch the target record via a GET request to obtain the latest ETag value, merge the local data, and retry with the updated If-Match header.
On-Premises Gateway Offline	Gateway health endpoint returns an unreachable status within the Power Platform Admin Center.	Configure HA Gateway Clustering (Active/Passive or Round Robin) during deployment ³⁹ . Trigger an alert to the internal infrastructure team.

File Size Limit Exceeded	On-Premises gateway returns a payload error indicating data is > 20MB during file creation ³ .	Implement automated pagination within the middleware export script, chunking the outbound CSV files into sequential 15 MB increments.
---------------------------------	---	---

Build-Ready Backlog

The following backlog organizes the critical development tasks required to execute the pilot, segregating requirements into immediate must-have stories and deferred enhancements.

Must-Have Stories for 4-6 Week Pilot

- **Story 1 (Infrastructure Provisioning):** As an M365 Administrator, I need an App Registration provisioned with the Sites.Selected Graph API permission scope, so that the custom middleware can securely authenticate to the specific target SharePoint site without violating least-privilege security principles.
- **Story 2 (OCR Pipeline Implementation):** As a Backend Engineer, I need to configure webhooks to transmit uploaded ticket images to Azure AI Document Intelligence, utilizing the prebuilt-invoice model to map the extracted JSON arrays into the predefined canonical ticket object schema.
- **Story 3 (Duplicate Detection Logic):** As a Backend Engineer, I need to implement an algorithm that generates an idempotency hash based on the concatenated slip number and dispatch timestamp, ensuring duplicate OCR scans are blocked from entering the database.
- **Story 4 (SharePoint Integration & Error Handling):** As a Backend Engineer, I need to write normalized ticket data to a SharePoint List using the Graph API SDK, implementing a robust exponential backoff protocol that parses and honors the Retry-After header to mitigate HTTP 429 throttling limits.
- **Story 5 (On-Premises Tunnel Configuration):** As an Infrastructure Engineer, I need to install, register, and verify the Microsoft On-Premises Data Gateway on a local Windows Server, establishing an outbound TCP 443 connection to enable scheduled flat-file drops to the internal ERP network folder.

Nice-to-Have Stories Deferred to Phase 2

- **Story 6 (High Availability Clustering):** As an Infrastructure Engineer, I need to install and configure a secondary On-Premises Data Gateway to establish a high-availability cluster, enabling failover redundancy for mission-critical file exports.
- **Story 7 (Automated PDF Invoicing):** As a Billing Specialist, I need the system to automatically trigger a Power Automate flow that populates a developer-configured Microsoft Word Template with SharePoint data to generate final PDF invoices. *(Deferred pending the acquisition and assignment of Power Automate Premium user licenses).*
- **Story 8 (Advanced Webhook Subscriptions):** As a Backend Engineer, I need to

subscribe to vendor-specific status webhooks (e.g., ticket.voided or ticket.updated) to automatically apply reversal logic or strike-through modifications in the SharePoint master ledger.

Evidence Requirements and Unknowns for Vendor Demo

During the technical validation and vendor demonstration phase, the engineering team must obtain explicit clarification on several abstracted technical parameters to finalize the integration design.

Explicit Unknowns Requiring Confirmation

1. **Rate Limits and Throttling Quotas:** What are the exact sustained and burst API rate limits (e.g., requests per second/minute) applied to the ticket ingestion and query endpoints for Command Alkon, Tread, and TruckIT?
2. **Native Idempotency Support:** Do the POST endpoints for ticket creation natively accept standard Idempotency-Key headers, or does the deduplication logic rely entirely on custom hashing on the client side?
3. **Pagination Strategy for Bulk Retrieval:** When querying bulk historical tickets, do the APIs utilize cursor-based pagination algorithms (e.g., passing a next_cursor token), offset/limit query parameters, or OData standard \$skiptoken structures?
4. **Webhook Delivery and Retry Guarantees:** Do the platform webhook implementations guarantee at-least-once delivery, and do they feature an automated retry schedule (e.g., exponential backoff over 24 hours) for unresponsive or offline receiver endpoints?
5. **Schema Customization and Truncation:** In platforms like HaulHub, does exceeding the stringent 100-character limit for HHCustom field names return a hard HTTP 400 rejection, or does the backend system gracefully truncate the string before ingestion?

Works cited

1. Populating a Word Template for an Easy Win - Lindsay T Shelton, https://www.lindsaytshelton.com/blog/20250112populating_a_word_template_for_an_easy_win/
2. Power Automate: "Power Apps Premium" vs Populate Word template to the Flow, <https://learn.microsoft.com/en-us/answers/questions/5827119/power-automate-power-apps-premium-vs-populate-word>
3. Limitation of file size creation using File System connector, <https://community.powerplatform.com/forums/thread/details/?threadid=9ff641c0-38bd-4bdf-851f-13936b6c30d1>
4. File System - Connectors | Microsoft Learn, <https://learn.microsoft.com/en-us/connectors/filesystem/>
5. Data Integration Process, <https://knowledge.haulhub.com/hubfs/connect/Data%20Integration%20Process.pdf>

6. TruckIT | Construction Dispatch & E-Ticketing Software, <https://truckit.com/>
7. Dump Truck Fleet Converts to Digital Ticketing with TruckIT, <https://www.worktruckonline.com/news/dump-truck-fleet-converts-to-digital-ticketing-with-truckit>
8. Blue Max Trucking Moves to Digital Ticketing | Work Truck Online, <https://www.worktruckonline.com/news/blue-max-trucking-moves-to-digital-ticketing>
9. Truckit API Documentation - Truckit.net, <https://developers.truckit.net/>
10. Booking - Truckit Apis - Zudoku, <https://developers.truckit.net/api/endpoints/booking>
11. HaulHub eTicket DOT API Integration Endpoint, <https://knowledge.haulhub.com/hubfs/connect/HaulHub%20eTicket%20DOT%20API.pdf>
12. HaulHub | Fast-Weigh Knowledge Base, <https://help.tacinsight.com/fast-weigh-knowledge-base/references/integrations/haulhub>
13. Command Cloud, <https://commandalkon.com/command-cloud/>
14. APIs - Command Alkon, <https://commandalkon.com/products/apis/>
15. Connected Partner Program - Command Alkon, <https://commandalkon.com/partner-program/>
16. command alkon incorporated product privacy documentation, <https://apac.cadev.deployedweb.com/product-privacy/>
17. Bringing Real-Time Analytics to Construction Logistics at Command Alkon - Medium, <https://medium.com/rocksetcloud/bringing-real-time-analytics-to-construction-logistics-at-command-alkon-e85bdb477442>
18. AI Native Forward Deployed Engineer @ Tread - Teal, https://www.tealhq.com/job/ai-native-forward-deployed-engineer_7ea1a60988cc109f28fc2727276db3619216b
19. Best Trucking Software 2026: 5-Tool Fleet Tech Stack - TenTrucks, <https://tentrucks.com/blog/best-trucking-software-2026-tech-stack>
20. Tread: Pricing, Free Demo & Features - Software Finder, <https://softwarefinder.com/fleet-management-software/tread>
21. How to check if file locked/open via Graph API/SDK - Stack Overflow, <https://stackoverflow.com/questions/71678626/how-to-check-if-file-locked-open-via-graph-api-sdk>
22. Microsoft Update DrivItem API throwing locked exception if the file is open in desktop, <https://stackoverflow.com/questions/61554807/microsoft-update-driveitem-api-throwing-locked-exception-if-the-file-is-open-in>
23. Need API to Determine if OneDrive/SharePoint Document Is Open or Locked, <https://learn.microsoft.com/en-au/answers/questions/5654593/need-api-to-determine-if-onedrive-sharepoint-docum>
24. Unable to Update or Delete SharePoint Library Record When File is Locked, <https://stackoverflow.com/questions/79749824/unable-to-update-or-delete-shar>

- [epoint-library-record-when-file-is-locked](#)
25. MS Sharepoint 2013 Development Questions and Answers,
<https://iq.iteanz.com/ms-sharepoint-2013-development-questions>
 26. Handling ETag value on SharePoint REST API Updates - ESPC,
<https://espc.tech/learning-hub/blog/handling-etag-value-on-sharepoint-rest-api-updates/>
 27. SharePoint REST APIs with HTTP Actions in Power Automate - Dynamics Services Group,
<https://dynamicservicesgroup.com/2025/12/30/sharepoint-rest-apis-with-http-actions-in-power-automate/>
 28. Microsoft Graph error responses and resource types,
<https://learn.microsoft.com/en-us/graph/errors>
 29. Microsoft Graph vs. SharePoint Online APIs (and where PnP fits): an exhaustive developer's guide - edvaldo b. guimarães filho,
<https://edvaldoguimaraes.com.br/2026/01/12/microsoft-graph-vs-sharepoint-online-apis-and-where-pnp-fits-an-exhaustive-developers-guide/>
 30. Sharepoint 2010 - Using the REST Interface - Google Sites,
<https://sites.google.com/site/icsharepoint2010/desarrollo/using-the-rest-interface>
 31. Avoid getting throttled or blocked in SharePoint Online - Microsoft Learn,
<https://learn.microsoft.com/en-us/sharepoint/dev/general-development/how-to-avoid-getting-throttled-or-blocked-in-sharepoint-online>
 32. Microsoft Graph throttling guidance,
<https://learn.microsoft.com/en-us/graph/throttling>
 33. Azure AI Document Intelligence for Invoice Extraction,
<https://invoicedataextraction.com/blog/azure-ai-document-intelligence-invoice-extraction>
 34. azure-ai-document-intelligence-dotnet - LobeHub,
<https://lobehub.com/skills/haniakrim21-everything-claude-code-azure-ai-document-intelligence-dotnet>
 35. Azure AI Document Intelligence | Azure AI Services Succinctly - Syncfusion,
<https://www.syncfusion.com/succinctly-free-ebooks/azure-ai-services-succinctly/azure-ai-document-intelligence>
 36. Word Online (Business) - Connectors - Microsoft Learn,
<https://learn.microsoft.com/en-us/connectors/wordonlinebusiness/>
 37. Populating Word doc template without needing premium license : r/PowerApps - Reddit,
https://www.reddit.com/r/PowerApps/comments/1q5gy2y/populating_word_doc_template_without_needing/
 38. Fill Word Templates without Premium Connectors in Power Automate — Guide | ESPC,
<https://espc.tech/learning-hub/blog/fill-word-templates-without-premium-connectors-in-power-automate-guide-%F0%9F%A4%96/>
 39. The On-Premises Data Gateway- The Most Underestimated Dependency in Every Power Platform Programme | VE3 Blog,

- <https://ve3.global/blog/the-on-premises-data-gateway--the-most-underestimated-dependency-in-every-power-platform-programme>
40. Power BI Gateway Architecture and High Availability Guide,
<https://powerbiconsulting.com/blog/power-bi-gateway-architecture-high-availability-2026>
 41. On-premises data gateway architecture | Microsoft Learn,
<https://learn.microsoft.com/en-us/data-integration/gateway/service-gateway-onprem-indepth>
 42. Manage on-premises data gateway high-availability clusters and load balancing,
<https://learn.microsoft.com/en-us/data-integration/gateway/service-gateway-high-availability-clusters>
 43. Clarifying the Scope of Files.ReadWrite.All Permission in Microsoft Graph API for SharePoint Access,
<https://learn.microsoft.com/en-us/answers/questions/1861330/clarifying-the-scope-of-files-readwrite-all-permissions>
 44. Moving from ACS & Password-Based SharePoint Access to Sites.Selected in Microsoft Graph | by Kathan Jain | Medium,
<https://medium.com/@kathanmecii/moving-from-acs-password-based-sharepoint-access-to-sites-selected-in-microsoft-graph-bfcdc5f77dd1>
 45. Microsoft Graph: Restricting SharePoint access beyond Sites.Selected,
<https://marvindickhaus.de/2026/01/powershell-graph-selectedoperations/>
 46. Overview of Selected Permissions in OneDrive and SharePoint - Microsoft Graph,
<https://learn.microsoft.com/en-us/graph/permissions-selected-overview>